

XRPL Validator — Daily Engineering Report • 2026-10-08

Generated 2026-10-09 00:21 EDT. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the live native-engine shadow, the reference rippled node, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **7 commits** (xrpl-validator 7)
- Live shadow: 3,759/3,759 ledgers fully matched, 350,263 transactions, 0 engine receipts in the current soak
- Amendment **PermissionDelegationV1_1** was enabled at ledger 107,524,865 (2026-10-08 21:29 UTC)
- Amendment **fixBatchV1_2** held majority (since 2026-09-25 14:12 UTC); activation due ~2026-10-09 14:12 UTC
- Amendment **BatchV1_1** held majority (since 2026-09-25 14:46 UTC); activation due ~2026-10-09 14:46 UTC

Live validator and native engine

Snapshot at generation time.

Measure	Value
Ledgers compared (current run)	3,759
Ledgers fully matched	3,759
Transactions applied natively	350,263
Result mismatches	0
Byte mismatches	0
Missing / extra keys	0 / 0
Batch inner-result mismatches	0
Engine receipts (current soak)	0
Live engine build running since	2026-10-08 20:14 EDT
Reference rippled	3.4.1, full, validated #107,531,247

Engineering changes

- **feat(node): Stage 4 Phase B — the native Rust engine writes state.rocks (XRPL_NATIVE_WRITER=1)** (4adb4a8 , xrpl-validator) On 2026-10-08 the network enabled PermissionDelegationV1_1. libxrpl 3.4.0 refused those transactions (temDISABLED); under Stage 3 its overlay was what got written, so #107525002 failed the state hash three times and the validator halted, while the native engine had applied them exactly as mainnet did.
- **feat(scripts): watch_engine says who writes state.rocks under the native writer** (5b6c81f , xrpl-validator) Under XRPL_NATIVE_WRITER the dashboard now has a "WHO WRITES THE LEDGER DATABASE" block: the Rust engine ACTIVELY WRITING (or LOADING, or OFF on the breaker), libxrpl COMPARING ONLY, and xrpld the reference node as the data feed plus the ledgers our engine could not vouch for. The FFI panel is

- titled COMPARISON ONLY, the native panel ACTIVELY WRITING / LOADING / WRITER OFF, and the writer's share counts ledgers since its copy of ...
- **feat(tools): xrpl-watch, the validator's terminal dashboard in Rust (HALCYON)** (d354cf0 , xrpl-validator) A ratatui successor to scripts/watch_engine.py, run beside the validator: tools/xrpl-watch , its own workspace so the validator's build and lock never see its dependencies.
 - **feat(tools): xrpl-watch shows every transaction type and result code** (45ca52a , xrpl-validator) The C++ panel lists the types and result codes since start, as many as fit in two lines each, ending "+N more · t". The t key opens a page with every type and every result code, counts and log-scaled bars (one tx is still visible next to forty thousand), tes / tec / other coloured apart; t again returns. --view types renders that page with --once and --record.
 - **feat(tools): xrpl-watch lists every type and result on the main screen** (e53ce35 , xrpl-validator) James: take out the event feed (or really reduce it) and the t page. The bottom row is now EVERY TX TYPE and EVERY RESULT CODE since the validator started, packed in columns with a log-scaled magnitude glyph per entry, plus THIS LEDGER; long names are shortened (Permissioned -> Perm, ...) before eliding. The event feed is one line, the newest event. The t page, the c key and --view are gone.
 - **feat(tools): xrpl-watch in real time — ~30 fps, the validator read 4x a second** (b945ba2 , xrpl-validator) James: "1 second refresh is fast but can seem kind of choppy". The screen now redraws at ~30 fps and reads the validator's API every 0.25 s (the upstream every 1 s, the process scan every 5 s, amendments every minute, the receipts file only when it changes). Between reads it animates: the counters roll to their new values, the pulse is a real-time trace (each ledger a beat where it closed, scrolling left and fading, ...
 - **feat(tools): xrpl-watch --record --fps, and a real-time footer** (c86e5e0 , xrpl-validator) --record takes --fps F: frames F times a second while the data is still read every --interval (the walkthrough video records 15 fps, reads 4x a second). The footer says what the screen now does: real time, ~30 fps, the validator read 4x a second.

Amendment watch

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Voting events:

- PermissionDelegationV1_1 was enabled — ledger 107,524,865, 2026-10-08 21:29 UTC

Amendment	Majority since	Activation due
fixBatchV1_2	2026-09-25 14:12 UTC	2026-10-09 14:12 UTC
BatchV1_1	2026-09-25 14:46 UTC	2026-10-09 14:46 UTC

Amendments enabled on mainnet: 95.

Appendix — commits

Time	Repo	Commit	Subject
18:52	xrpl-validator	4adb4a8	feat(node): Stage 4 Phase B — the native Rust engine writes state.rocks (XRPL_NATIVE_WRITER=1)
20:20	xrpl-validator	5b6c81f	feat(scripts): watch_engine says who writes state.rocks under the native writer
20:51	xrpl-validator	d354cf0	feat(tools): xrpl-watch, the validator's terminal dashboard in Rust (HALCYON)

Time	Repo	Commit	Subject
21:17	xrpl-validator	45ca52a	feat(tools): xrpl-watch shows every transaction type and result code
21:21	xrpl-validator	e53ce35	feat(tools): xrpl-watch lists every type and result on the main screen
21:25	xrpl-validator	b945ba2	feat(tools): xrpl-watch in real time — ~30 fps, the validator read 4x a second
21:46	xrpl-validator	c86e5e0	feat(tools): xrpl-watch --record --fps, and a real-time footer